

Experiment 4: Ensemble Prediction and Decision Tree Model Evaluation

Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An autonomous Institution affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch:2023-2028	Due date:
Name	Ponsubash Raj R	Register No.	3122237001037

Aim

To use the characteristics of the cell nuclei present in the digitized image of a fine needle aspirate (FNA) of a breast mass and identify the existence of Breast Cancer.

Objective

To build classifiers such as Decision Tree, AdaBoost, Gradient Boosting, XGBoost, Random Forest, and Stacked Models (using SVM, Naïve Bayes, Decision Tree) and evaluate their performance through 5-Fold Cross-Validation and hyperparameter tuning.

Libraries Used

- numpy
- pandas
- matplotlib
- scikit-learn

Theoretical Description

1. Decision Tree

Decision Tree is a **non-parametric supervised learning algorithm** that partitions the input space into regions based on feature values. At each node, the algorithm selects the feature that maximizes information gain (or minimizes impurity).

Impurity Measures:

- Gini Index: $G = 1 - \sum_{k=1}^K p_k^2$
- Entropy: $H = - \sum_{k=1}^K p_k \log_2(p_k)$

The split criterion:

$$\text{Information Gain}(A) = H(\text{parent}) - \sum_{i=1}^m \frac{n_i}{n} H(\text{child}_i)$$

Decision is made by traversing from root to leaf using learned rules.

2. AdaBoost (Adaptive Boosting)

AdaBoost is an **ensemble method** that combines multiple weak learners (often decision stumps) into a strong classifier by iteratively adjusting sample weights.

At iteration t :

$$\begin{aligned}\epsilon_t &= \frac{\sum_{i=1}^n w_i \mathbb{I}(y_i \neq h_t(x_i))}{\sum_{i=1}^n w_i} \\ \alpha_t &= \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) \\ w_i &\leftarrow w_i \exp(-\alpha_t y_i h_t(x_i))\end{aligned}$$

Final strong classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

3. Gradient Boosting

Gradient Boosting builds an ensemble sequentially by fitting new learners to the **residual errors** (negative gradients) of the previous model.

Objective function:

$$\mathcal{L}(y, F(x)) = \sum_{i=1}^n \ell(y_i, F(x_i))$$

At each stage m :

$$\begin{aligned}r_{im} &= - \left[\frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}} \\ h_m(x) &\approx r_{im} \quad (\text{fit weak learner}) \\ F_m(x) &= F_{m-1}(x) + \eta h_m(x)\end{aligned}$$

where η is the learning rate.

4. XGBoost (Extreme Gradient Boosting)

XGBoost is an **optimized implementation of Gradient Boosting** that adds **regularization, shrinkage, and parallelization**.

The objective function:

$$\mathcal{L}(t) = \sum_{i=1}^n \ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

Regularization term:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

where:

- T = number of leaves,
 - γ = complexity penalty,
 - λ = L2 regularization on leaf weights.
-

5. Random Forest

Random Forest is an **ensemble of decision trees** trained on bootstrapped samples using **bagging** and **feature randomness**.

Steps:

- Sample training data with replacement (bootstrap).
- For each split, randomly choose a subset of features.
- Grow multiple decision trees in parallel.

Final prediction:

$$\hat{y} = \text{majority vote} \quad (\text{classification}), \quad \hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(x) \quad (\text{regression})$$

6. Stacking (SVM + Naive Bayes + Decision Tree)

Stacking is an **ensemble learning method** that combines predictions from multiple base models (**level-0**) using a **meta-learner (level-1)**.

Process:

- Train base models: SVM, Naive Bayes, and Decision Tree.
- Collect predictions of base models as new features.

- Train a meta-classifier (e.g., Logistic Regression) on these features.

Mathematically:

$$z_i = [h_1(x_i), h_2(x_i), \dots, h_M(x_i)]$$

$$\hat{y}_i = g(z_i)$$

where h_m are base learners, and g is the meta-learner.

Python Code

```
"""# IMPORT FUNCTIONS"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import roc_curve, roc_auc_score

from sklearn.model_selection import train_test_split,
    GridSearchCV, StratifiedKFold
from sklearn.metrics import accuracy_score, precision_score,
    recall_score, f1_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler,
    Binarizer, LabelEncoder

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier,
    RandomForestClassifier, StackingClassifier
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
import time
from sklearn.metrics import confusion_matrix

from xgboost import XGBClassifier

"""# CONFIGURATIONS"""

TARGET_COL = "class"
UNNECESSARY_COLS = []
CATEGORICAL_FILL_MODE = "mode"
NUMERIC_FILL_VALUES = {}
OUTLIER_FILTER = {}
GAUSSIAN_LIKE = ['radius1', 'texture1', 'perimeter1', 'smoothness1',
    'symmetry1', 'texture3', 'smoothness3']
```

```

# fetch dataset
breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)

# data (as pandas dataframes)
X = breast_cancer_wisconsin_diagnostic.data.features
y = breast_cancer_wisconsin_diagnostic.data.targets

y = y.squeeze()
if not np.issubdtype(y.dtype, np.number):
    le = LabelEncoder()
    y = pd.Series(le.fit_transform(y), name="diagnosis")

df = pd.concat([X.reset_index(drop=True), y.reset_index(drop=True)
                .rename(TARGET_COL)], axis=1)

option = int(input(
    "Choose the model:\n"
    "1. Decision Tree\n"
    "2. AdaBoost\n"
    "3. Gradient Boosting\n"
    "4. XGBoost\n"
    "5. Random Forest\n"
    "6. Stacking (SVM + Naive Bayes + Decision Tree)\n"
))

stack_estimators = [
    ("svm", make_pipeline(StandardScaler(),
                          SVC(probability=True, kernel="rbf"))),
    ("nb", GaussianNB()),
    ("dt", DecisionTreeClassifier(random_state=42)),
]

xgb_model = XGBClassifier(
    objective="binary:logistic",
    eval_metric="logloss",
    use_label_encoder=False,
    random_state=42
)

model_param_map = {
    1: {
        "model": DecisionTreeClassifier(random_state=42),
        "param_grid": {
            "max_depth": [None, 3, 5, 7, 10],
            "min_samples_split": [2, 5, 10]
        }
    },
    2: {

```

```

        "model": AdaBoostClassifier(random_state=42),
        "param_grid": {
            "n_estimators": [50, 100, 200],
            "learning_rate": [0.5, 1.0, 1.5]
        }
    },
    3: {
        "model": GradientBoostingClassifier(random_state=42),
        "param_grid": {
            "n_estimators": [100, 200],
            "learning_rate": [0.05, 0.1],
            "max_depth": [2, 3]
        }
    },
    4: {
        "model": xgb_model,
        "param_grid": (
            {
                "n_estimators": [100, 200],
                "max_depth": [3, 5],
                "learning_rate": [0.05, 0.1],
                "subsample": [0.8, 1.0],
                "colsample_bytree": [0.8, 1.0]
            }
        )
    },
    5: {
        "model": RandomForestClassifier(random_state=42),
        "param_grid": {
            "n_estimators": [100, 200],
            "max_depth": [None, 5, 10],
            "min_samples_split": [2, 5]
        }
    },
    6: {
        "model": StackingClassifier(
            estimators=stack_estimators,
            final_estimator=LogisticRegression(max_iter=1000, random_state=42),
            n_jobs=-1
        ),
        "param_grid": {
            # Tune SVM C inside the stacked pipeline + tree depth
            "svm__svc__C": [0.1, 1, 10],
            "dt__max_depth": [None, 5, 10]
        }
    },
}

```

```

if option not in model_param_map:
    raise ValueError("Invalid option selected!")

model_config = model_param_map[option]
base_model = model_config["model"]
param_grid = model_config["param_grid"]

if UNNECESSARY_COLS:
    df.drop(columns=[col for col in UNNECESSARY_COLS if col in df.columns],
            inplace=True)

"""# FILL MISSING VALUES"""

for col in df.columns:
    count = df[col].isna().sum()
    print(f"{col} -> {count} missing values")

"""# SPLITTING TO FEATURES AND TARGETS"""

X = df.drop(columns=[TARGET_COL])
y = df[TARGET_COL]

X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.3,
        stratify=y, random_state=42)

if option == 6: # Stacking model uses SVM and scaling is required
    numerical_cols = df.select_dtypes(include=['int64', 'float64'])
        .columns.tolist()
    features_to_scale = [col for col in numerical_cols if col != TARGET_COL]

    scaler = StandardScaler()
    X_temp[GAUSSIAN_LIKE] = scaler.fit_transform(X_temp[GAUSSIAN_LIKE])

    non_gaussian = [col for col in features_to_scale if col not in GAUSSIAN_LIKE]
    normalizer = MinMaxScaler()
    X_temp[non_gaussian] = normalizer.fit_transform(X_temp[non_gaussian])

X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.5,
        stratify=y_temp, random_state=42)

"""# CROSS VALIDATION"""

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid = GridSearchCV(estimator=base_model,
        param_grid=param_grid,
        scoring='accuracy',

```

```

        cv=skf,
        n_jobs=-1)
grid.fit(X_val, y_val)

print("Best hyperparameters:", grid.best_params_)

best_model = grid.best_estimator_

cv_metrics = {
    "Fold": [], "Accuracy": [], "Precision": [], "Recall": [], "F1": []
}

fold = 1
for train_idx, val_idx in skf.split(X_train, y_train):
    X_train_fold, X_val_fold = X_train.iloc[train_idx], X_train.iloc[val_idx]
    y_train_fold, y_val_fold = y_train.iloc[train_idx], y_train.iloc[val_idx]

    best_model.fit(X_train_fold, y_train_fold)
    y_pred = best_model.predict(X_val_fold)

    cv_metrics["Fold"].append(f"Fold {fold}")
    cv_metrics["Accuracy"].append(accuracy_score(y_val_fold, y_pred))
    cv_metrics["Precision"].append(precision_score(y_val_fold, y_pred,
        zero_division=0))
    cv_metrics["Recall"].append(recall_score(y_val_fold, y_pred))
    cv_metrics["F1"].append(f1_score(y_val_fold, y_pred))
    fold += 1

# Add average row
cv_metrics["Fold"].append("Average")
for metric in ["Accuracy", "Precision", "Recall", "F1"]:
    cv_metrics[metric].append(np.mean(cv_metrics[metric]))

print("\n Cross-Validation Metrics:")
print(pd.DataFrame(cv_metrics))

"""# Validation"""

# Best model from GridSearchCV
best_model = grid.best_estimator_

# Measure training time
start_time = time.time()
best_model.fit(X_train, y_train)
training_duration = time.time() - start_time
print(f"\n Model Training Time: {training_duration:.2f} seconds\n")

def evaluate_classification(name, y_true, y_pred):

```



```

print(f"\n{name} Metrics:")
print(f"  Accuracy : {accuracy_score(y_true, y_pred):.4f}")
print(f"  Precision: {precision_score(y_true, y_pred, zero_division=0):.4f}")
print(f"  Recall   : {recall_score(y_true, y_pred):.4f}")
print(f"  F1-Score : {f1_score(y_true, y_pred):.4f}")

if name == "Test":
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
    plt.title(f'{name} - Confusion Matrix')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.tight_layout()
    plt.show()

if option == 6:
    X_test[GAUSSIAN_LIKE] = scaler.transform(X_test[GAUSSIAN_LIKE])
    X_test[non_gaussian] = normalizer.transform(X_test[non_gaussian])

for name, (X_split, y_split) in zip(["Train", "Validation", "Test"],
    [(X_train, y_train), (X_val, y_val), (X_test, y_test)]):

    y_pred = best_model.predict(X_split)
    evaluate_classification(name, y_split, y_pred)

    if name == "Test":

        if hasattr(best_model, "predict_proba"):
            y_probs = best_model.predict_proba(X_split)[: , 1]
        elif hasattr(best_model, "decision_function"):
            y_probs = best_model.decision_function(X_split)
        else:
            print("Model does not support ROC plotting.")
            continue

        # Plot ROC Curve
        fpr, tpr, _ = roc_curve(y_split, y_probs)
        auc_score = roc_auc_score(y_split, y_probs)

        plt.figure(figsize=(8, 6))
        plt.plot(fpr, tpr, label=f"AUC = {auc_score:.4f}")
        plt.plot([0, 1], [0, 1], "k--", label="Random Guess")
        plt.xlabel("False Positive Rate")
        plt.ylabel("True Positive Rate")
        plt.title("ROC Curve")
        plt.legend(loc="lower right")
        plt.grid()

```

```
plt.tight_layout()
plt.show()
```

Hyper Parameter tuning

Decision Tree Model

Table 1: Decision Tree - Hyperparameter Tuning

criterion	max_depth	Accuracy	F1 Score
gini	None	0.9144	0.8844
gini	3	0.9194	0.8879
gini	5	0.9244	0.8958
gini	7	0.9144	0.8844
entropy	None	0.9296	0.9034
entropy	3	0.9245	0.8930
entropy	5	0.9346	0.9103
entropy	7	0.9296	0.9034
log_loss	None	0.9296	0.9034
log_loss	3	0.9245	0.8930
log_loss	5	0.9346	0.9103
log_loss	7	0.9296	0.9034

Adaboost Model

Table 2: AdaBoost - Hyperparameter Tuning

learning_rate	n_estimators	Accuracy	F1 Score
0.5	50	0.9595	0.9425
0.5	100	0.9545	0.9365
0.5	200	0.9445	0.9232
1.0	50	0.9545	0.9352
1.0	100	0.9495	0.9287
1.0	200	0.9495	0.9287
1.5	50	0.9395	0.9148
1.5	100	0.9496	0.9295
1.5	200	0.9495	0.9287

Gradient Boosting Model

Table 3: Gradient Boosting - Hyperparameter Tuning

learning_rate	max_depth	n_estimators	Accuracy	F1 Score
0.05	2	100	0.9395	0.9175
0.05	2	200	0.9345	0.9115
0.05	3	100	0.9245	0.8975
0.05	3	200	0.9245	0.8975
0.1	2	100	0.9345	0.9115
0.1	2	200	0.9295	0.9036
0.1	3	100	0.9245	0.8975
0.1	3	200	0.9194	0.8891

XGBoost Model

Table 4: XGBoost - Hyperparameter Tuning

gamma	learning_rate	max_depth	n_estimators	Accuracy	F1 Score
0	0.05	3	100	0.9546	0.9389
0	0.05	3	200	0.9495	0.9323
0	0.05	5	100	0.9546	0.9389
0	0.05	5	200	0.9445	0.9249
0	0.1	3	100	0.9495	0.9323
0	0.1	3	200	0.9445	0.9262
0	0.1	5	100	0.9546	0.9386
0	0.1	5	200	0.9546	0.9386
1	0.05	3	100	0.9546	0.9389
1	0.05	3	200	0.9546	0.9389
1	0.05	5	100	0.9546	0.9389
1	0.05	5	200	0.9546	0.9389
1	0.1	3	100	0.9546	0.9389
1	0.1	3	200	0.9546	0.9389
1	0.1	5	100	0.9496	0.9328
1	0.1	5	200	0.9496	0.9328

Random Forest Model

Table 5: Random Forest - Hyperparameter Tuning

criterion	max_depth	n_estimators	Accuracy	F1 Score
gini	None	100	0.9496	0.9312
gini	None	200	0.9496	0.9312
gini	5	100	0.9496	0.9312
gini	5	200	0.9445	0.9249
gini	10	100	0.9496	0.9312
gini	10	200	0.9496	0.9312
entropy	None	100	0.9495	0.9291
entropy	None	200	0.9495	0.9291
entropy	5	100	0.9496	0.9312
entropy	5	200	0.9496	0.9312
entropy	10	100	0.9495	0.9291
entropy	10	200	0.9495	0.9291
log_loss	None	100	0.9495	0.9291
log_loss	None	200	0.9495	0.9291
log_loss	5	100	0.9496	0.9312
log_loss	5	200	0.9495	0.9291
log_loss	10	100	0.9495	0.9291
log_loss	10	200	0.9495	0.9291

Stacked Ensemble Model

Table 6: Stacked Ensemble - Hyperparameter Tuning

Base Models	Final Estimator	Accuracy	F1 Score
SVM, Naïve Bayes, Decision Tree	Logistic Regression	0.9532	0.9344
SVM, Naïve Bayes, Decision Tree	Random Forest	0.9591	0.9431
SVM, Decision Tree, KNN	Logistic Regression	0.9649	0.9508

Table 7: 5-Fold Cross Validation Results for All Models

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average Accuracy
Decision Tree	0.925	0.875	0.85	0.975	0.8974	0.9044
AdaBoost	0.95	0.85	0.925	0.975	0.8974	0.9194
Gradient Boosting	0.95	0.875	0.85	1.0	0.8717	0.9093
XGBoost	0.95	0.925	0.95	1.0	0.8717	0.9393
Random Forest	0.925	0.90	0.925	1.0	0.8974	0.9294
Stacked Model	0.95	0.925	0.975	0.925	0.9487	0.9447

Plots

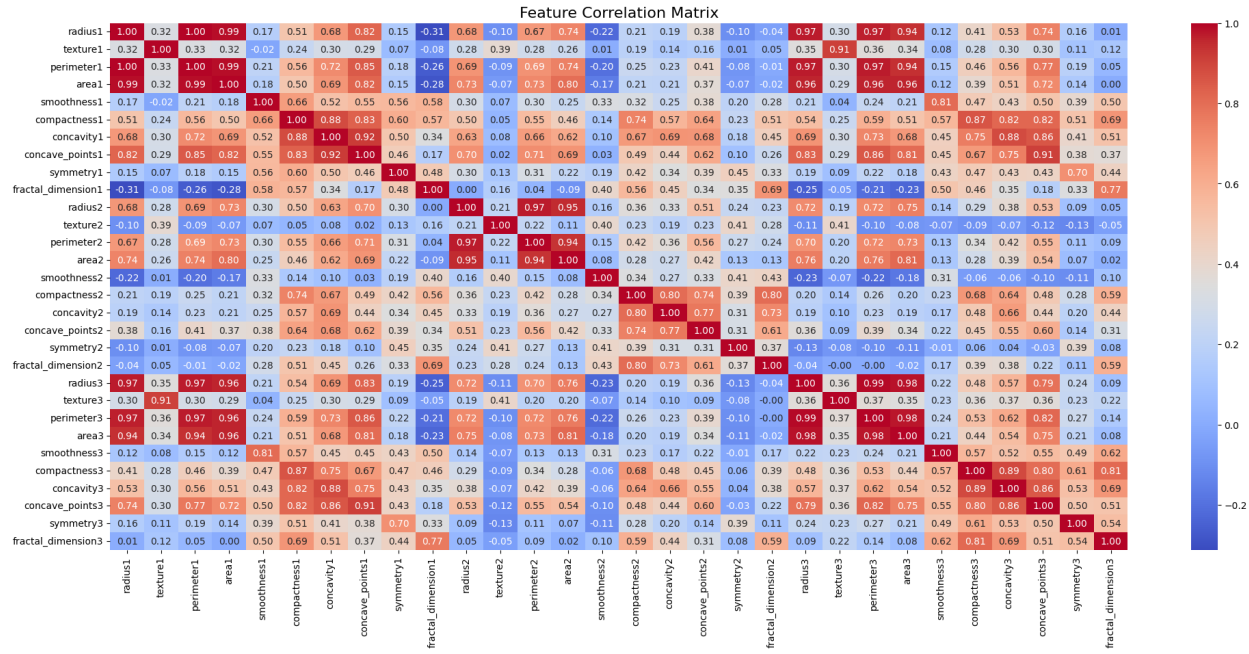
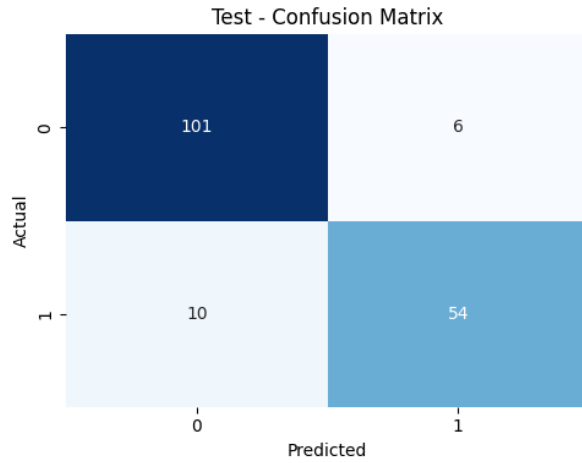
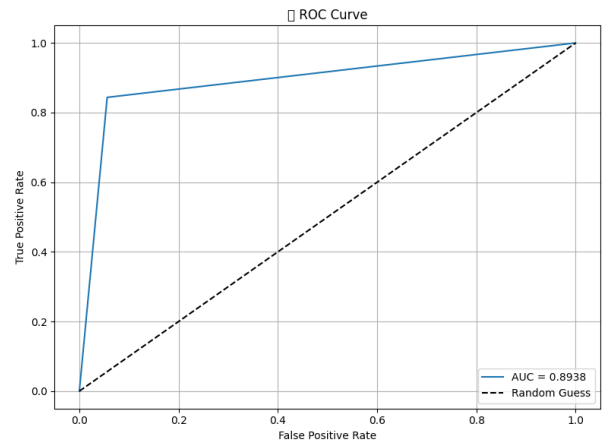


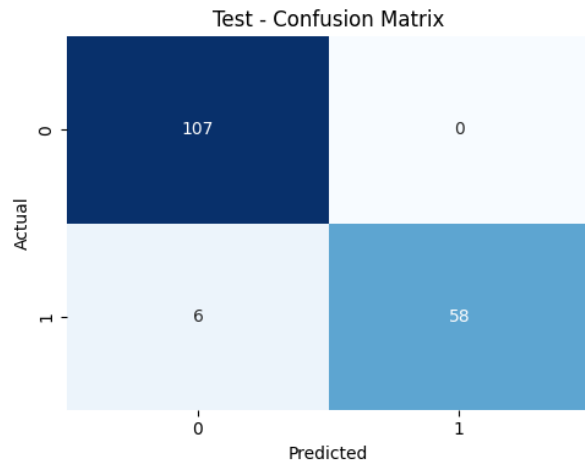
Figure 1: Correlation Heatmap



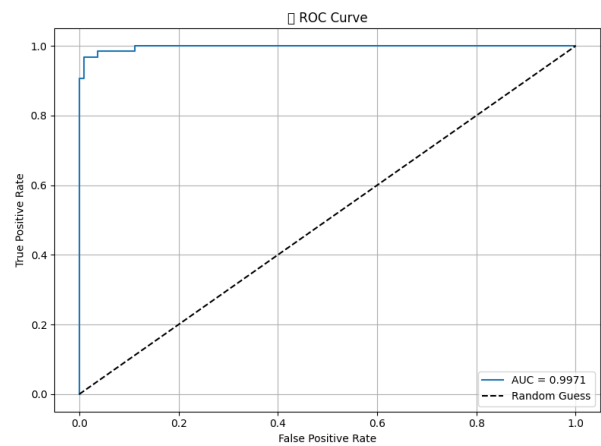
(a) Decision Tree Confusion Matrix



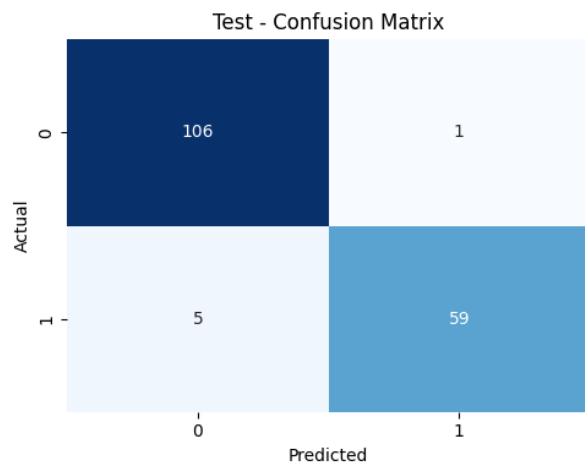
(b) Decision Tree ROC Curve



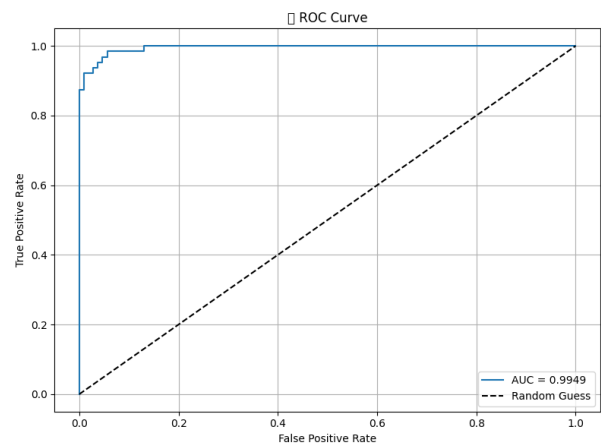
(a) Adaboost Confusion Matrix



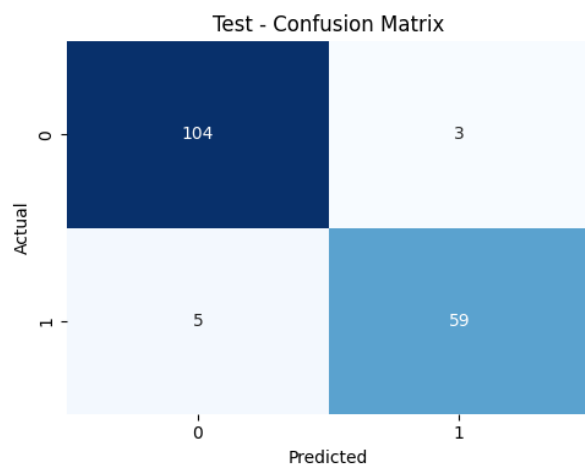
(b) Adaboost ROC Curve



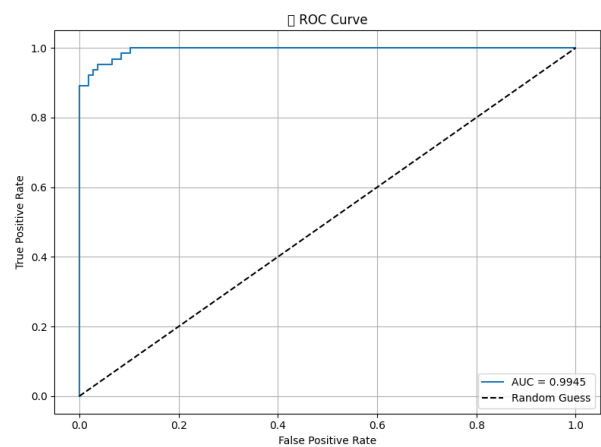
(a) Gradient Boosting Confusion Matrix



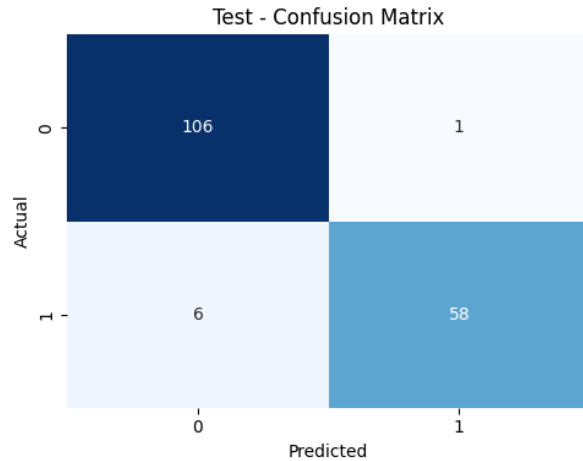
(b) Gradient Boosting ROC Curve



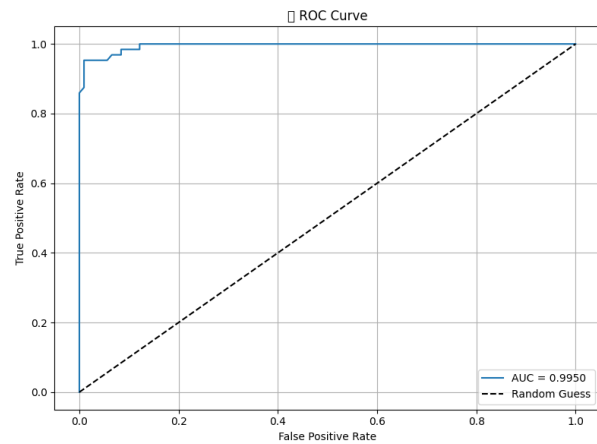
(a) XG-Boost Confusion Matrix



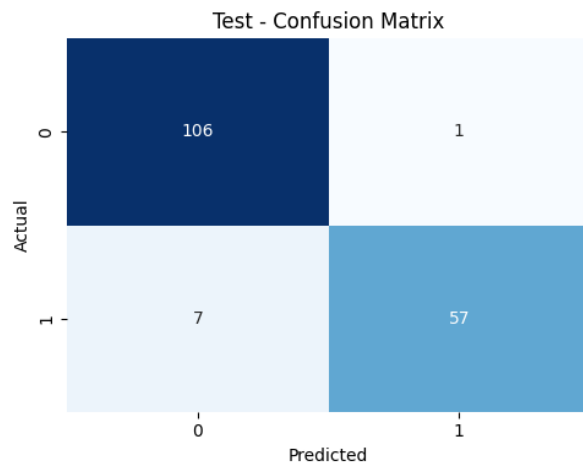
(b) XG-Boost ROC Curve



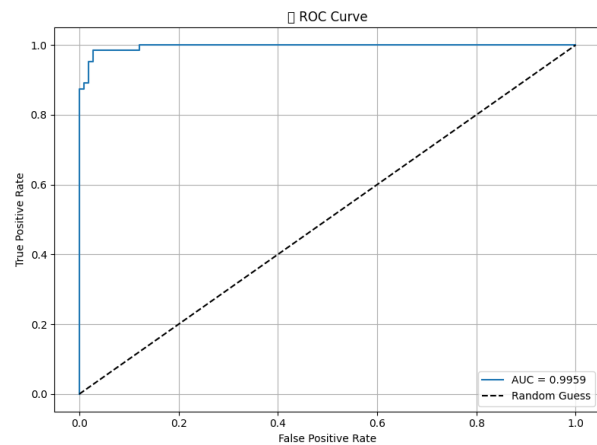
(a) Random Forest Confusion Matrix



(b) Random Forest ROC Curve



(a) Stacked Model Confusion Matrix



(b) Stacked Model ROC Curve

Observation Questions

1. **Which model achieved the best validation accuracy among all six methods?**

The Stacked model achieved the best accuracy, especially the SVM + Decision Tree + KNN stacked model with Logistic Regression final estimator performed the best with an accuracy of 0.9649.

2. **How does Decision Tree performance compare to ensemble methods?**

A single Decision Tree usually has lower accuracy and tends to overfit. The Decision Tree achieved an accuracy of 0.90, while ensemble methods achieved accuracies around 0.95–0.96, which is significantly better.

3. **Did the Random Forest benefit from tuning max_depth or n_estimators?**

There was no significant benefit from tuning max_depth or n_estimators, as it gave consistent performance of about 0.94 across all hyper-parameter combinations.

4. **Which model showed the best generalization? Any overfitting?**

Overfitting was observed clearly in the Decision Tree Classifier, attaining 100% accuracy on training data but only 90% accuracy on test data. XGBoost and Stacked models had the best generalization and gave similar metrics for both train and test data.

5. Did stacking improve performance over base models?

There was only a very slight improvement in performance, but nothing significant. The best base models like XGBoost and Random Forest achieved 0.95 accuracy, while the best stacked model achieved 0.96 accuracy.

Conclusion

- Learnt to implement Decision Tree algorithm, ensemble algorithms like Adaboost, Gradient Boosting, XG Boost, Random Forest and stacking different models to improve the performance.
- Able to perform hyperparameter tuning and use the best set of hyper-parameters to decide the final model metrics.